Evaluating Mamba Against Persistence for Financial Time-Series Forecasting: Evidence from Gold and Exchange-Rate Series

Bing Mo[*1], Wei Zhang[1], Xianzhong Zhou[†2], Qi Zheng[‡3,4], and Feng Yang[4]

[1]Frontier Interdisciplinary Domain, Beijing Institute of Technology, Zhuhai, China
[2]School of Integrated Circuits, Guangdong University of Technology, Guangzhou, China
[3]Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, China
[4]Silictec (Shenzhen) Electronic Technology Co., Ltd., Shenzhen, China

Abstract

Selective state-space models are increasingly used for time-series forecasting, but their performance relative to tuned attention, linear, decomposition, patch-based, and persistence methods remains protocol- and dataset-dependent. This descriptive study evaluates four Mamba variants, ten non-Mamba neural baselines, and a deterministic last-value forecast on two univariate financial level series: the GLD unadjusted close and a United Kingdom exchange-rate series. The neural comparison comprises 336 model–horizon test cells, supplemented by 24 last-value cells, across 12 forecast lengths and chronological 70/10/20 splits. Within the completed neural sweeps, hyperparameters are selected on validation data; the reported test scores are nevertheless treated as development-period evidence because earlier GLD test outcomes informed architecture promotion. On GLD, PMS-Mamba-PD records balanced RMSSE 14.74, below TimesNet at 15.66 and last-value at 17.07. On Exchange, last-value is lowest at 6.26, followed by S-Mamba at 6.29 and PatchTST at 6.37; thus, no learned model improves on persistence for that series under the current protocol. These are single-seed point estimates from overlapping forecast windows, with unequal search budgets and no inferential intervals. The results support conditional competitiveness of the tested Mamba variants on GLD, not general superiority or a confirmatory financial-forecasting claim.

1 Introduction

Long-horizon forecasting requires a model to extract predictive structure while the usefulness of past observations changes with the forecast length. Modern approaches address this problem through efficient attention, decomposition, temporal patches, inverted tokenization, and multi-scale representations [1, 2, 3, 4, 5, 6]. Their relative performance is strongly protocol- and dataset-dependent: lookback length, normalization, optimization budget, temporal split, and evaluation metric can alter model rankings [7, 8, 9].

Nonstationarity adds a second difficulty. Changes in level, scale, trend, volatility, or local dynamics can make future windows differ from the observations used for training. RevIN addresses window-level distribution shift through reversible normalization, while Non-stationary Transformers combine stationarization with a learned mechanism intended to restore useful nonstationary

[*]Corresponding author: hitmbzq@126.com
[†]Corresponding author: zhouxzh@gdut.edu.cn
[‡]Corresponding author: mbzq@163.com

information [10, 11]. Decomposition models provide a complementary inductive bias by separating slowly varying and residual components before forecasting [2, 12, 3].

Mamba offers input-selective state-space sequence modeling with linear scaling in sequence length [13]. Forecasting adaptations differ in how they tokenize time and variables, scan in one or two directions, and combine temporal resolutions [14, 15, 16, 17, 18]. Recent multi-scale and decomposition-enhanced Mamba studies make this design space increasingly crowded [19, 20, 21]. An efficient or more elaborate sequence operator is therefore not, by itself, evidence of improved forecasting.

The requirement for a strong reference forecast is particularly important for persistent financial level series. Forecasting competitions repeatedly show that simple methods can remain difficult to beat [7, 8], while financial forecasting reviews emphasize sensitivity to target definition, regime changes, and evaluation design [22]. A model comparison on price or exchange-rate levels should therefore include a last-observation forecast before attributing an observed difference to architectural sophistication.

This study reports the completed repository benchmark for four Mamba configurations: S-Mamba, a decomposition variant (NS-Mamba-D), a multi-scale decomposition variant (MS-Mamba-MD), and a patch-temporal multi-scale decomposition variant (PMS-Mamba-PD). They are compared with ten tuned neural baselines and a deterministic last-value forecast on GLD and Exchange under single-target (S-mode) forecasting. Both datasets use 12 forecast lengths from 1 to 360 observations. Within each final neural sweep, candidate configurations are selected by validation performance and evaluated on the test period. However, earlier GLD test results were consulted during architecture development; consequently, the present evaluation is descriptive rather than an untouched confirmatory test.

The contributions are:

- a traceable two-series benchmark covering 14 trained models, a deterministic last-value reference, and 12 forecast lengths;
- an implementation-grounded comparison of inverted, decomposition, multi-scale, and patch-temporal Mamba configurations against diverse neural alternatives;
- a reproducible persistence calculation showing that the strongest learned model differs by series and that last-value is best on Exchange under the current metric; and
- an explicit account of test-period reuse, single-seed uncertainty, overlapping forecast windows, search-budget asymmetry, post hoc dataset exclusion, and univariate tokenization.

2 Related Work

2.1 Long-Horizon Forecasting

Neural forecasting architectures encode temporal structure in substantially different ways. Informer and Reformer reduce the computational cost of attention [1, 23]; Autoformer and FEDformer incorporate decomposition [2, 12]; and DLinear shows that a small decomposition-based linear model can remain competitive [3]. PatchTST uses temporal patches, TimesNet maps temporal variation into two-dimensional representations, and iTransformer represents variables rather than time points as tokens [4, 5, 6]. N-BEATS and N-HiTS provide further examples of basis expansion and explicit multi-rate structure without attention [24, 25]. These alternatives motivate a diverse comparison rather than one restricted to Transformer variants.

2.2 Nonstationarity and Distribution Shift

Forecasting under distribution shift has motivated both normalization and architecture-level responses. RevIN normalizes each input instance and reverses the transformation at the output [10]. Non-stationary Transformers argue that normalization can remove predictive information and introduce a learned de-stationary mechanism [11]. Decomposition methods separate trend-like and residual structure so that one representation need not explain all temporal scales [2, 12, 3]. The variants evaluated here combine reversible scaling with decomposition, multiple downsampling rates, or temporal patches, but their component labels denote architectural constructions rather than uniquely identified physical processes.

2.3 Mamba for Time Series

Mamba replaces self-attention with an input-selective state-space recurrence [13]. S-Mamba applies bidirectional Mamba blocks over variable tokens for forecasting [14]; Bi-Mamba+ studies bidirectionality and token relations [15]; MambaTS examines temporal and variable scan strategies [16]; and TimeMachine combines multiple Mamba pathways under channel-mixing and channel-independent settings [17]. TSMamba further explores transfer through a bidirectional time-series foundation model [18].

Several later studies overlap directly with the decomposition and multi-scale ideas considered in this repository. ms-Mamba uses Mamba blocks operating at multiple temporal sampling rates [19]; KARMA combines multilevel decomposition with multi-scale Mamba blocks for multivariate forecasting [20]; and DMamba assigns different forecasting modules to decomposed components [21]. The present study does not claim priority for decomposition or multi-scale Mamba as general principles. Its narrower purpose is to document the behavior of four implemented configurations under one repository protocol and to expose where that evidence is insufficient.

2.4 Evaluation Perspective

Forecasting competitions show that rankings vary across series, horizons, and evaluation choices, and that simple reference methods remain necessary comparators [7, 8]. Accordingly, this study retains a last-value forecast, reports each dataset separately, and does not average raw errors across targets with different scales. The reported model minima and horizon profiles are descriptive test-period summaries, not statistically significant differences. This distinction matters because the current runs use one seed, overlapping forecast windows, nonidentical optimization budgets, and a test period that was observed during architecture development.

3 Forecasting Models

3.1 Task Definition

Let $\mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L \times C}$ be a lookback window of length L and let $\mathbf{Y}_{t+1:t+H} \in \mathbb{R}^{H \times C_o}$ be the forecast target. All experiments use single-target S mode, so $C = C_o = 1$. Both tasks use

$$H \in \{1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360\}, \tag{1}$$

Each horizon has a separately trained model and can use a model-specific lookback L selected on validation data.

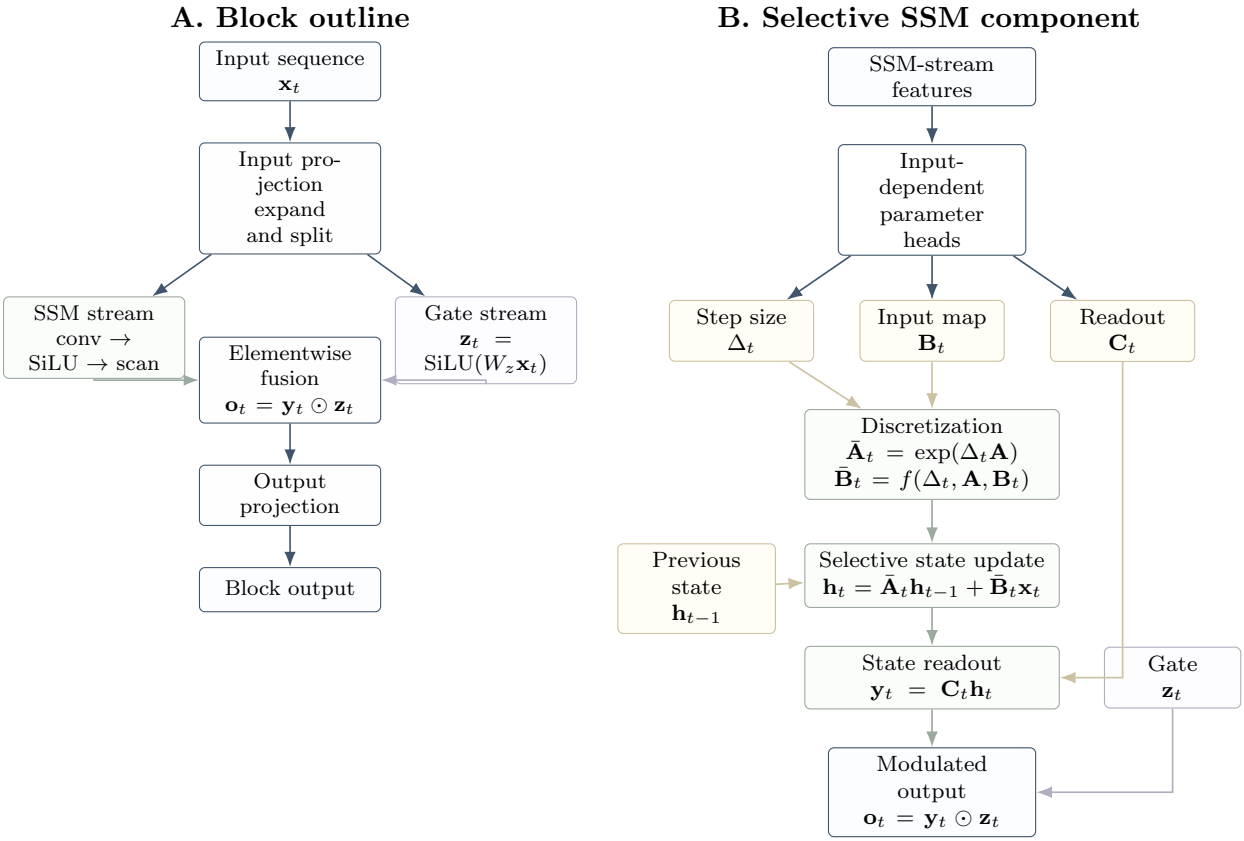


Figure 1: Conceptual Mamba block. Input-dependent selective state-space parameters control the recurrent update and readout before gated fusion and output projection.

3.2 Selective State-Space Backbone

A linear state-space model maps input $\mathbf{x}_t$ to latent state $\mathbf{h}_t$ and output $\mathbf{y}_t$ through

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}\mathbf{h}_t. \tag{2}$$

Mamba makes the discretization and readout input-dependent, yielding selective parameters $\bar{\mathbf{A}}_t$, $\bar{\mathbf{B}}_t$, and $\mathbf{C}_t$ [13]:

$$\mathbf{h}_t = \bar{\mathbf{A}}_t\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}_t\mathbf{h}_t. \tag{3}$$

Figure 1 summarizes the sequence block used by the Mamba-family implementations.

3.3 S-Mamba Forecasting Encoder

S-Mamba first applies reversible window normalization [14]. For sample b and variable c,

$$\mu_{b,c} = \frac{1}{L}\sum_{t=1}^{L} X_{b,t,c}, \qquad \sigma_{b,c} = \sqrt{\frac{1}{L}\sum_{t=1}^{L}(X_{b,t,c} - \mu_{b,c})^2 + \epsilon}, \tag{4}$$

$$Z_{b,t,c} = \frac{X_{b,t,c} - \mu_{b,c}}{\sigma_{b,c}}. \tag{5}$$

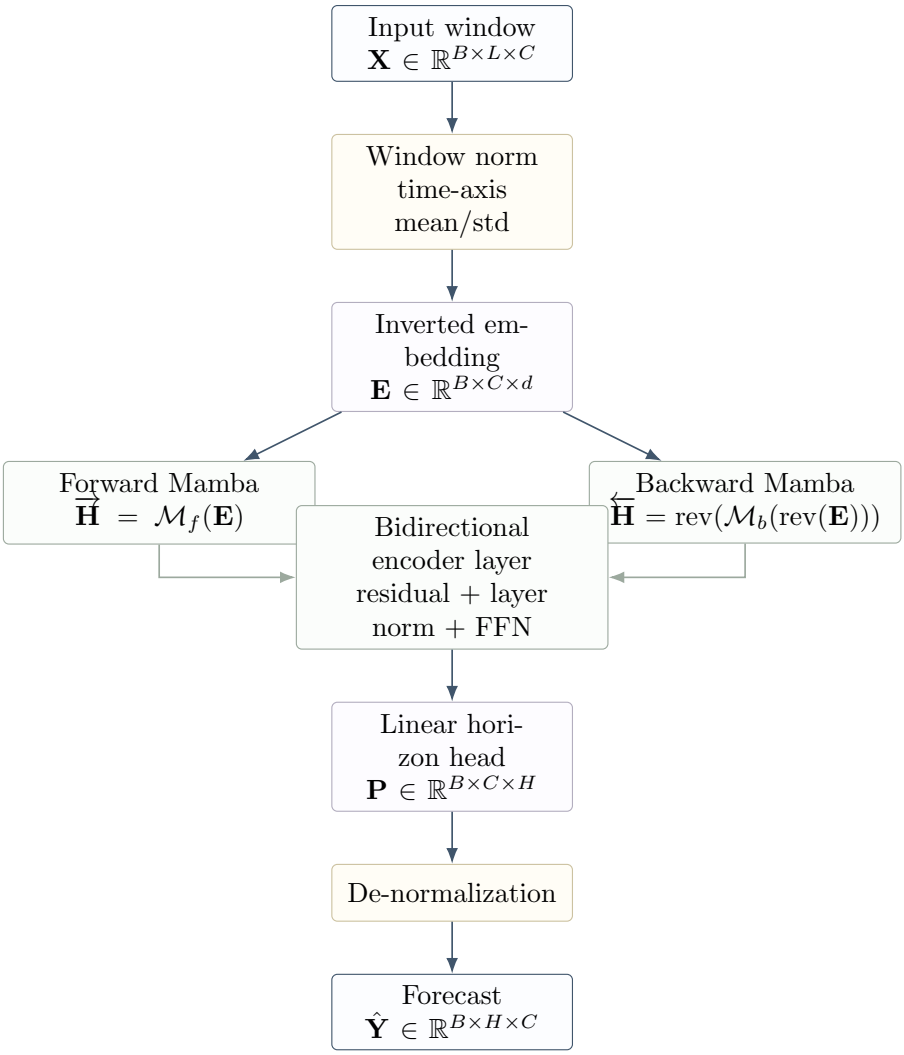


Figure 2: S-Mamba forecasting encoder with reversible normalization, variable-token embedding, bidirectional Mamba encoding, horizon projection, and inverse normalization.

The complete normalized history of each variable becomes one token:

$$\mathbf{e}_{b,c} = W_{\mathrm{emb}}\mathbf{z}_{b,:,c} + \mathbf{b}_{\mathrm{emb}}, \qquad \mathbf{E} \in \mathbb{R}^{B\times C\times d_{\mathrm{model}}}. \tag{6}$$

A bidirectional encoder applies independent Mamba scans to the token order and its reversal, sums the two outputs, and then applies residual connections, layer normalization, and a feed-forward block. The final token representation is projected to H values and transformed back to the original target scale:

$$\hat{Y}_{b,h,c} = \sigma_{b,c}P_{b,c,h} + \mu_{b,c}. \tag{7}$$

In the present S-mode benchmark, $C = 1$ and the inverted encoder therefore receives one variable token. The bidirectional variable scan cannot represent cross-variable dependence in this setting. This is a property of the experimental task, not a change to the implementation, and it motivates the multivariate limitation discussed later.

3.4 Decomposition Variant

NS-Mamba-D applies standard reversible normalization and separates the normalized window into low-frequency, medium-frequency, and residual components. Let $\mathrm{MA}_m(\cdot)$ denote a centered moving average with boundary replication and selected window length m:

$$\mathbf{Z}^{\mathrm{tr}} = \mathrm{MA}_m(\mathbf{Z}), \tag{8}$$

$$\mathbf{R}^0 = \mathbf{Z} - \mathbf{Z}^{\mathrm{tr}}, \tag{9}$$

$$\mathbf{Z}^{\mathrm{mf}} = \mathrm{MA}_m(\mathbf{R}^0), \tag{10}$$

$$\mathbf{Z}^{\mathrm{re}} = \mathbf{R}^0 - \mathbf{Z}^{\mathrm{mf}}. \tag{11}$$

Three S-Mamba branches forecast the components independently and their normalized outputs are summed:

$$\hat{\mathbf{Y}}_{\mathrm{D}} = \mathcal{N}^{-1}\left(\sum_{q\in\{\mathrm{tr},\mathrm{mf},\mathrm{re}\}} \mathcal{F}_{\theta_q}(\mathbf{Z}^q)\right), \tag{12}$$

where $\mathcal{N}$ is reversible normalization and $\mathcal{F}_{\theta_q}$ is a forecasting branch. The component labels describe the architectural bias; they are not asserted to be uniquely identifiable physical processes.

3.5 Multi-Scale and Patch-Temporal Variants

MS-Mamba-MD adds average-pooled temporal scales. For scale set $\mathcal{S}$ and downsampling operator $\mathcal{P}_s$, the implementation concatenates the three component series along the variable dimension,

$$\mathbf{V}_s = \mathcal{P}_s\left(\mathrm{concat}_C[\mathbf{Z}^{\mathrm{tr}}, \mathbf{Z}^{\mathrm{mf}}, \mathbf{Z}^{\mathrm{re}}]\right). \tag{13}$$

A single scale-specific inverted branch $\mathcal{F}_{\theta_s}$ processes all $3C$ component tokens; there is not an independently parameterized branch for every component. Its output is reshaped by component and summed,

$$\mathbf{Q}_s = \mathcal{F}_{\theta_s}(\mathbf{V}_s), \qquad U_{s,c,h} = \sum_{q=1}^{3} Q_{s,q,c,h}. \tag{14}$$

The selected MD configuration then averages the scale forecasts,

$$\hat{\mathbf{Y}}_{\mathrm{MD}} = \mathcal{N}^{-1}\left(\frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}} \mathbf{U}_s\right). \tag{15}$$

PMS-Mamba-PD retains the same concatenated component representation and shared branch per scale, but replaces the inverted branch with a temporal patch-token branch. For component channel k, patch length p, and stride r, the branch embeds patch j as

$$\mathbf{e}_{j,k}^{(s)} = W_p\,\mathrm{vec}\left(\mathbf{V}_{s,j:j+p-1,k}\right), \tag{16}$$

applies Mamba over the patch index, projects each component channel to H values, and sums the three component outputs before mean fusion across scales. Unlike the one-token S-Mamba encoder in S mode, this branch presents a temporal token sequence to the selective state-space operator.

Table 1: Mamba configurations included in the completed benchmark.

Variant	Active structure	Fusion
S-Mamba	Inverted variable-token backbone	Single path
NS-Mamba-D	Three-way decomposition	Component sum
MS-Mamba-MD	Decomposition and temporal scales	Mean over scales
PMS-Mamba-PD	Decomposition, scales, and temporal patches	Mean over scales

3.6 Deterministic Reference Forecast

The last-value reference repeats the most recent observation available at forecast origin o for every lead h:

$$\hat{Y}^{\mathrm{LV}}_{o,h} = X_o, \qquad h = 1, \ldots, H. \tag{17}$$

It has no trainable parameters, lookback selection, or validation step. The reference is evaluated on exactly the same test origins and lead-time aggregation used for the learned models.

3.7 Training Objective

All models minimize mean squared error on the scaled training target:

$$\mathcal{L} = \frac{1}{BHC_o} \sum_{b=1}^{B} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left(\hat{Y}_{b,h,c} - Y_{b,h,c} \right)^2 . \tag{18}$$

Inverse transformation is enabled for evaluation, so saved predictions, targets, and reported errors are on each target's original scale.

4 Experimental Setup

4.1 Datasets and Forecasting Tasks

Table 2 summarizes the two included S-mode tasks. Only the named target column is provided to each model, even when the source file contains additional variables. Both files are ordered, contain no duplicate dates, and contain no missing values in the evaluated target. Horizons are reported in observations because GLD follows business-day trading observations and the processed Exchange file contains only non-missing source dates.

Table 2: Included univariate forecasting tasks. Split counts follow the chronological 70/10/20 loader rule.

Dataset	Target	Sampling	Total	Train	Validation	Test	Horizons
GLD	`close`	Business-day observations	5,395	3,776	540	1,079	1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360
Exchange	`OT` (United Kingdom)	Non-missing daily quotes	13,885	9,719	1,389	2,777	1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360

GLD was downloaded from Yahoo Finance with ticker `GLD` and `auto_adjust=False`; the target is therefore the unadjusted close rather than an adjusted return series [26]. The local snapshot spans 2004-11-18 to 2026-04-30. Its training, validation, and test targets end on 2019-11-18, 2022-01-10, and 2026-04-30, respectively.

The Exchange source is a repository-local long-format compilation identified as originating from FRED [27]. The preparation script `scripts/prepare_exchange_uk.py` selects rows labelled

`United Kingdom`, removes missing source values, sorts by date, and renames the value to `OT`. The resulting series spans 1971-01-04 to 2026-05-15, with target boundaries at 2009-09-17, 2015-04-06, and 2026-05-15. The raw retrieval record and exact FRED series identifier were not preserved; the quote direction and unit therefore cannot be independently verified from the repository and the target is treated as a quoted rate rather than assigned an economic unit.

Validation and test datasets include up to `seq_len` earlier observations as input context, but their forecast targets remain within the nominal split. The loader fits a `StandardScaler` on the training target only and applies it to all splits. Predictions and targets are inverse-transformed before metrics are computed.

Weather was run exploratorily before a data-quality defect was identified: 50 training-target values encoded as `-9999` were neither replaced nor masked before scaling and training. Its exclusion is therefore a post hoc protocol amendment made after exploratory outcomes existed, not a pre-specified dataset filter. No Weather value contributes to the reported comparison; a documented cleaning rule and complete rerun of every method are required before it can be used as evidence.

4.2 Metrics, Origins, and Aggregation

For a nominal forecast length H, each test example predicts all leads $h = 1, \ldots, H$. With T_{test} target observations, the sliding-window loader yields $O_H = T_{\text{test}} - H + 1$ origins and $N_H = O_H H$ scalar forecast errors. The reported value for length H pools every origin and all leads up to H; it is not an error calculated only at lead H. Thus, origin counts range from 1,079 to 720 for GLD and from 2,777 to 2,418 for Exchange as H increases from 1 to 360.

For predictions $\hat{Y}$ and targets Y, root mean squared error is

$$\text{RMSE} = \sqrt{\frac{1}{N_H} \sum_{i=1}^{N_H} (Y_i - \hat{Y}_i)^2}. \tag{19}$$

RMSSE normalizes forecast error by one-step changes in the inverse-transformed training target $X_{1:T}$ [8]:

$$\text{RMSSE} = \sqrt{\frac{N_H^{-1} \sum_{i=1}^{N_H} (Y_i - \hat{Y}_i)^2}{(T-1)^{-1} \sum_{t=2}^{T} (X_t - X_{t-1})^2}}. \tag{20}$$

The denominator is fixed within a dataset but differs across datasets, so raw RMSSE is compared only within each dataset.

Short, middle, and long sets are $\{1, 3, 5, 7\}$, $\{14, 30, 60, 90\}$, and $\{120, 180, 270, 360\}$. If $\bar{R}_g$ denotes mean RMSSE within group g, the study-level aggregation is

$$\text{BalancedRMSSE} = \frac{1}{3} \left(\bar{R}_{\text{short}} + \bar{R}_{\text{middle}} + \bar{R}_{\text{long}} \right). \tag{21}$$

Because every group contains four forecast lengths here, this quantity is also the arithmetic mean of the 12 RMSSE values. It is a protocol-specific summary and does not create independent replications from the 12 overlapping tasks.

4.3 Training and Hyperparameter Selection

Each learned model–horizon configuration is trained separately with Adam, at most 10 epochs, early-stopping patience 3, and configured seed 2023. Optuna uses a seeded TPE sampler and median pruning. The neural baseline family contains Autoformer, DLinear, Informer, NLinear,

Non-stationary Transformer, PatchTST, Reformer, TimesNet, Transformer, and iTransformer. The Mamba family contains S-Mamba, NS-Mamba-D, MS-Mamba-MD, and PMS-Mamba-PD. Last-value is deterministic and receives no tuning budget.

The neural search first evaluates a calibration horizon set. Up to five top completed trials are replayed across the full validation horizon suite, and the candidate with the lowest validation balanced RMSSE is selected. Only that selected configuration is evaluated by the final-sweep code on the test split. This procedure yields 336 trained-model test cells, 168 per dataset; the deterministic calculation adds 24 last-value cells.

Search budgets are not identical. GLD stores 20 trial records per baseline and Mamba model, but its refreshed neural-baseline search calibrates on all 12 horizons while the Mamba search calibrates on three. Exchange stores 20 neural-baseline trial records and 24 Mamba trial records per model. Search spaces are model-specific, and some archived studies contain interrupted or failed states. Table 5 reports the exact records; comparisons are therefore model-specific tuned point estimates rather than equal-compute optimization results.

The split discipline above describes each final sweep, not the complete research cycle. Repository development records show that earlier GLD test outcomes informed promotion of decomposition and patch-temporal variants before the reported benchmark was frozen. Accordingly, the current GLD test period is development evidence and cannot support an untouched confirmatory claim.

4.4 Result and Software Provenance

The four trained-model result matrices follow `results/<dataset>_<phase>_optuna_best.csv`; selected configurations and trial histories are stored under `results/optuna/`. The last-value rows are deterministically regenerated from the two target files by `manuscript/scripts/generate_benchmark_results.py` and written to `manuscript/data/last_value_baseline.csv`. The same generator checks model–horizon coverage, completion state, split flags, metric finiteness, group aggregation, and agreement with selected-parameter metadata before writing the tables.

All 216 Exchange and GLD-Mamba trained rows match saved run-level `metrics.npy` files. The 120 GLD neural-baseline rows are frozen in the authoritative CSV and a reproduction bundle containing parameters, checkpoints, predictions, and replay summaries. The pinned software environment records Python 3.9.23, PyTorch 2.6.0 with CUDA 12.6 build, `mamba-ssm` 2.2.4, NumPy 1.23.5, pandas 2.3.1, and scikit-learn 1.6.1. The original GPU model, runtime, memory use, and energy consumption were not recorded, so no computational-efficiency comparison is made.

5 Results

5.1 Dataset-Level Point Estimates

Table 3: Descriptive balanced-RMSSE point estimates on the current test periods. The neural and Mamba entries are the post hoc within-family minima; Last-value is deterministic and requires no fitting. No uncertainty interval is available.

Dataset	Last-value	Neural baseline	RMSSE	Mamba	RMSSE	Lowest point estimate
GLD	17.07	TimesNet	15.66	PMS-Mamba-PD	14.74	PMS-Mamba-PD (14.74)
Exchange	6.26	PatchTST	6.37	S-Mamba	6.29	Last-value (6.26)

Table 3 changes the interpretation of the neural-only comparison by retaining persistence. On GLD, PMS-Mamba-PD records balanced RMSSE 14.74, compared with 15.66 for TimesNet and

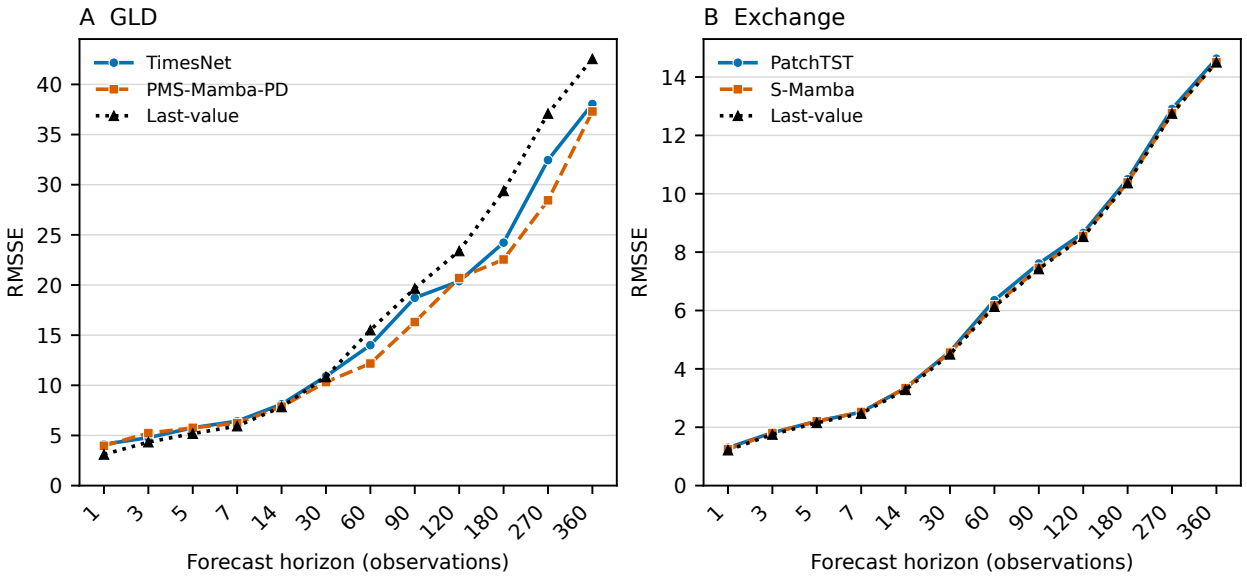


Figure 3: RMSSE by nominal forecast length for one fixed method from each comparison class. Within each dataset, the displayed neural and Mamba models are selected descriptively by the lowest test-period balanced RMSSE and are then held fixed across all 12 lengths: TimesNet and PMS-Mamba-PD for GLD; PatchTST and S-Mamba for Exchange. Last-value is deterministic. Each point pools all test origins and leads $1, \ldots, H$. Values are single-run point estimates without uncertainty intervals.

17.07 for last-value. These point estimates correspond to a 5.89% reduction relative to TimesNet and a 13.66% reduction relative to last-value. They show that the selected PMS-Mamba-PD run is competitive on this GLD test period, but they do not quantify sampling or training uncertainty.

On Exchange, last-value records the lowest balanced RMSSE, 6.26. S-Mamba is the lowest learned model at 6.29, approximately 0.58% higher than last-value, while PatchTST records 6.37. Consequently, the earlier neural-only statement that S-Mamba improves on the strongest baseline does not survive inclusion of the deterministic reference.

Figure 3 avoids a per-horizon family oracle: every learned curve represents one model across the complete length set. RMSSE increases with nominal forecast length for all displayed methods because each length pools progressively more distant leads. The GLD curves separate more visibly at intermediate and long lengths, whereas the Exchange curves remain close and the persistence curve is lowest in the aggregate.

5.2 Model-Level Comparison

Table 4: Balanced-RMSSE point estimates for all evaluated methods. Neural models are validation-selected single-seed runs; Last-value is a deterministic reference. Bold marks the lowest observed value in each dataset. No cross-dataset rank is aggregated.

Model	Type	GLD	Exchange
Last-value	Reference	17.07	**6.26**
MS-Mamba-MD	Mamba	15.98	6.35
NS-Mamba-D	Mamba	15.24	6.30
PMS-Mamba-PD	Mamba	**14.74**	6.31
S-Mamba	Mamba	16.79	6.29
Autoformer	Neural baseline	17.45	6.62
DLinear	Neural baseline	18.78	7.32
Informer	Neural baseline	75.35	8.18
NLinear	Neural baseline	16.48	6.42
Non-stationary Transformer	Neural baseline	19.96	6.59
PatchTST	Neural baseline	15.99	6.37
Reformer	Neural baseline	74.01	13.45
TimesNet	Neural baseline	15.66	6.61
Transformer	Neural baseline	72.42	8.19
iTransformer	Neural baseline	15.68	6.56

Table 4 reports every evaluated method without pooling ranks across the two targets. Among Mamba variants, PMS-Mamba-PD is lowest on GLD, while S-Mamba is lowest on Exchange; NS-Mamba-D is close to the selected Mamba model on both series. No Mamba configuration is lowest on both datasets once last-value is included. The observed ordering therefore supports a dataset-specific description, not a universal architectural ranking.

6 Discussion and Limitations

The descriptive result differs by series. On GLD, the selected PMS-Mamba-PD point estimate is below both TimesNet and last-value, whereas persistence is lowest on Exchange. The Exchange result is especially important for interpretation: a small neural-to-neural difference does not establish forecasting value when neither learned model improves on the last observation. The evidence therefore supports conditional competitiveness on one target period rather than a general Mamba advantage.

The present test periods are not fully independent confirmation sets for the research program. Although the final sweep implementation selects hyperparameters on validation data, earlier GLD test outcomes informed architecture promotion. This research-cycle reuse can make the final GLD

ordering optimistic and cannot be corrected through textual qualification alone. A confirmatory study requires a newly frozen future period or an additional untouched dataset, with the model family, metric, exclusions, and analysis fixed before outcomes are inspected.

Optimization and stochastic uncertainty further limit architecture comparisons. Exchange allocates 24 trial records per Mamba study and 20 per neural-baseline study, while GLD uses different calibration suites across families. Search spaces and pruning rates are model-specific, and several studies retain interrupted or failed records. Every selected configuration uses seed 2023, and CUDA execution is not guaranteed to be bitwise deterministic. In the GLD reproduction bundle, 90 of 120 replayed neural-baseline metric vectors match the frozen table at strict numerical tolerance, while TimesNet balanced RMSSE changes from 15.66 to 15.50. The resulting PMS-Mamba-PD reduction is 4.94% rather than 5.89%, illustrating why small differences require repeated training.

The test errors are also statistically dependent. Adjacent forecast origins overlap, and every nominal length H pools leads 1 through H; estimates at different origins, leads, and lengths are therefore not independent replications. The 360 reported method–length cells summarize evaluation tasks rather than a sample size for inference. Repeated seeds and paired uncertainty procedures that respect temporal dependence, such as block resampling over forecast origins or an appropriately specified loss-differential analysis, are needed before making inferential claims.

The S-mode design limits mechanism claims. S-Mamba's inverted encoder normally scans variable tokens, but each experiment here contains only one variable token, so the bidirectional variable scan cannot express cross-variable dependence. PMS-Mamba-PD instead scans temporal patch tokens, but the current comparison does not isolate tokenization, decomposition, parameter count, or optimization budget in a controlled ablation. The results therefore do not demonstrate that a particular component causes an improvement. Likewise, absent parameter, runtime, memory, and hardware records, Mamba's computational scaling is a literature motivation rather than an empirical result of this study.

Dataset scope remains narrow. Two financial level series and one chronological split do not represent the diversity of forecasting applications, and no rolling-origin evaluation is included. The study predicts levels rather than returns and reports statistical errors rather than direction, risk, transaction-cost-adjusted utility, or another economic outcome. It should not be interpreted as evidence of trading profitability or market inefficiency. Weather cannot currently broaden the evidence because its exclusion followed exploratory runs with an unhandled missing-value code; all methods must be rerun after a prespecified cleaning rule.

Finally, the Exchange extraction is reproducible from the local raw file, but the original FRED series identifier, quote convention, and retrieval record are missing. The exact experiment hardware is also absent, and the working repository has not yet been archived as a citable release. These provenance gaps must be resolved before a reproducibility claim extends beyond the accompanying local artifacts.

7 Conclusion

This study describes completed single-seed evaluations of four Mamba variants, ten tuned neural baselines, and last-value forecasts on two univariate financial level series. On GLD, PMS-Mamba-PD has the lowest observed balanced RMSSE at 14.74, compared with TimesNet at 15.66 and last-value at 17.07. On Exchange, last-value is lowest at 6.26, ahead of S-Mamba at 6.29 and PatchTST at 6.37.

The benchmark therefore does not establish general Mamba superiority. Its defensible result is narrower: the tested patch-temporal Mamba configuration is competitive on the current GLD

period, while persistence remains the stronger reference on Exchange. Because architecture development used prior GLD test outcomes, search budgets differ, training uses one seed, forecast windows overlap, and no inferential intervals are available, all reported differences are descriptive. An untouched evaluation period, equal-compute tuning, repeated seeds, dependence-aware uncertainty, controlled ablations, and broader datasets are required before confirmatory or mechanism-level claims can be made.

A Appendix

A.1 Optimization Records

Table 5: Archived Optuna study records. The state column gives complete/pruned/other trial counts across all model studies in a row. Each study replays up to five top completed calibration trials on the full validation horizon set before one final test evaluation.

Dataset	Family	Trial records/model	Models	Trial states	Calibration horizons
GLD	Baseline	20	10	124/76/0	1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360
GLD	Mamba	20	4	37/43/0	7, 90, 360
Exchange	Baseline	20	10	129/69/2	7, 90, 360
Exchange	Mamba	24	4	52/44/0	7, 90, 360

The archived state counts in Table 5 are reported instead of assuming that every requested trial completed. The baseline and Mamba studies use the search spaces implemented in `scripts/optuna/run_dataset_optuna.py`. Shared choices include lookback length, learning rate, batch size, and model width where applicable. Baseline-specific choices include decomposition windows, patch and stride sizes, attention factors, normalization, and family-specific widths. Mamba-specific choices include state dimension, decomposition window, scale set, and patch/stride pairs.

The exact selected parameters, study names, calibration objectives, validation replay scores, and final test scores are stored in:

- the GLD directory at `results/optuna/gld/`;
- the Exchange directory at `results/optuna/exchange/`.

The associated baseline and Mamba trial CSV files retain every archived Optuna state and per-horizon calibration metric.

A.2 Evaluated Mamba Configurations

Four Mamba labels enter the completed benchmark. S-Mamba is the reversible-normalized inverted forecasting backbone. NS-Mamba-D adds three-way moving-average decomposition and sums component forecasts. MS-Mamba-MD adds multiple downsampled temporal scales with mean scale fusion. PMS-Mamba-PD replaces the inverted branch at each scale with a temporal patch-token Mamba branch while retaining decomposition and mean fusion. Other architecture labels described by the implementation are not included in the 336 trained-model cells and are not used to support the reported comparison.

A.3 Deterministic Reference and Dataset Preparation

The manuscript result generator computes last-value errors using the same 70/10/20 split, test origins, all-lead pooling, and training-difference RMSSE denominator as the learned models. It writes 24 rows to `manuscript/data/last_value_baseline.csv`, including the origin count and scalar forecast count for each nominal length. No fitted parameter or test-informed choice enters this calculation.

The current Exchange target can be checked against its local long-format input with

```
conda run -n mamba python scripts/prepare_exchange_uk.py --check
```

The script selects the `United Kingdom` rows, drops missing quoted values, sorts the dates, and verifies exact equality with `data/Exchange/exchange_rate.csv`.

A.4 Evidence and Regeneration

The manuscript build reads the four final trained-model matrices directly from `results/` and the two target files from `data/`. Running `make -C manuscript pdf` first regenerates the last-value rows, Table 3, Table 4, Table 5, and Figure 3. Generation fails if a trained model–horizon row is missing, a run is incomplete, selection/test flags are wrong, a metric is non-finite, balanced RMSSE does not match its term groups, a final score disagrees with selected-parameter metadata, or a last-value target is missing.

The GLD baseline reproduction bundle is rooted at `scripts/gld_baseline_optuna_bundle/`; its run archive is the `reproduced_artifacts` subdirectory. It contains checkpoints, predictions, targets, metrics, and run metadata for all 120 baseline rows. Because stochastic replay changes 30 metric vectors, the frozen `results/gld_baseline_optuna_best.csv` remains the source for the descriptive GLD table; the replay difference is disclosed in the limitations rather than silently replacing the selected-run result.

The repository currently lacks a public archival tag or DOI containing the exact manuscript state, and the original experiment hardware record is incomplete. These items remain prerequisites for an external reproducibility claim and formal submission package.

References

[1] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, 2021.

[2] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.

[3] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.

[4] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.

[5] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis. In *International Conference on Learning Representations*, 2023.

[6] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.

[7] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The m4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74, 2020.

[8] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364, 2022.

[9] Jose F. Torres, Dalil Hadjout, Abderrazak Sebaa, Francisco Martinez-Alvarez, and Alicia Troncoso. Deep learning for time series forecasting: A survey. *Big Data*, 9(1):3–21, 2021.

[10] Tae-Hyun Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2022.

[11] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Advances in Neural Information Processing Systems*, 2022.

[12] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, 2022.

[13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[14] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Xiaocui Yang, Han Zhao, Daling Wang, and Yifei Zhang. Is mamba effective for time series forecasting? *Neurocomputing*, 619:129178, 2025.

[15] Aobo Liang, Xingguo Jiang, Yan Sun, Xiaohou Shi, and Ke Li. Bi-mamba+: Bidirectional mamba for time series forecasting. *arXiv preprint arXiv:2404.15772*, 2024.

[16] Xiuding Cai, Yaoyao Zhu, Xueyao Wang, and Yu Yao. Mambats: Improved selective state space models for long-term time series forecasting. *arXiv preprint arXiv:2405.16440*, 2024.

[17] Md Atik Ahamed and Qiang Cheng. Timemachine: A time series is worth 4 mambas for long-term forecasting. In *European Conference on Artificial Intelligence*, 2024.

[18] Haoyu Ma, Yushu Chen, Wenlai Zhao, Jinzhe Yang, Yingsheng Ji, Xinghua Xu, Xiaozhu Liu, Hao Jing, Shengzhuo Liu, and Guangwen Yang. A mamba foundation model for time series forecasting. *arXiv preprint arXiv:2411.02941*, 2024.

[19] Yusuf Meric Karadag, Ismail Talaz, Ipek Gursel Dino, and Sinan Kalkan. ms-mamba: Multi-scale mamba for time-series forecasting. *arXiv preprint arXiv:2504.07654*, 2025.

[20] Hang Ye, Gaoxiang Duan, Haoran Zeng, Yangxin Zhu, Lingxue Meng, Xiaoying Zheng, and Yongxin Zhu. KARMA: A multilevel decomposition hybrid mamba framework for multivariate long-term time series forecasting. *arXiv preprint arXiv:2506.08939*, 2025.

[21] Ruxuan Chen and Fang Sun. DMamba: Decomposition-enhanced mamba for time series forecasting. *arXiv preprint arXiv:2602.09081*, 2026.

[22] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing*, 90:106181, 2020.

[23] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[24] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020.

[25] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza, Max Mergenthaler-Canseco, and Artur Dubrawski. N-hits: Neural hierarchical interpolation for time series forecasting. In *AAAI Conference on Artificial Intelligence*, 2023.

[26] Yahoo Finance. SPDR Gold Shares (GLD) Historical Data. https://finance.yahoo.com/quote/GLD/history/. Repository snapshot ends 2026-04-30.

[27] Federal Reserve Bank of St. Louis. FRED: Federal Reserve Economic Data. https://fred.stlouisfed.org/. Repository source snapshot ends 2026-05-15.